BFCPU

October 30, 2023

Contents

1 Introduction **3**

2 Brainf*ck **4**
- 2.1 BF as a Language . . . 4
- 2.2 Architectures . . . 5
  - 2.2.1 Von Neumann . . . 5
  - 2.2.2 Harvard . . . 5
  - 2.2.3 BF Architecture . . . 6
- 2.3 BF as an Instruction Set . . . 6
- 2.4 Brainfix . . . 6

3 Architecture **7**
- 3.1 Overview . . . 7
- 3.2 Instruction Pointer Register (IP) . . . 8
- 3.3 Stack Pointer (SP) . . . 8
- 3.4 Instruction Register (I) . . . 9
- 3.5 Data Register (D) and Data Pointer Register (DP) . . . 9
- 3.6 Flags and the Flag Register (F) . . . 9
  - 3.6.1 A and V . . . 9
  - 3.6.2 Z(D) and Z(LS) . . . 10
- 3.7 Loop Skip Register (LS) . . . 10
  - 3.7.1 Preprocessing . . . 10
- 3.8 Control Unit and Control Signals . . . 11
  - 3.8.1 Control Unit . . . 11
  - 3.8.2 Register Control Signals . . . 11
  - 3.8.3 Memory (RAM) . . . 12
  - 3.8.4 Screen (Output) . . . 12
  - 3.8.5 Keyboard (Input) . . . 12

4 Control Sequences **14**
4.1 Cylce 0 14
4.2 Modifying Data: `+` and `-` 15
4.3 Moving the Pointer: `<` and `>` 15
4.4 Conditional Jumping: `[` and `]` 15
4.5 Output: 16
4.6 Input: `,` and `'` 16
4.7 Non-BF instructions: `NOP` and `ERR` 17

5 Implementation **21**
5.1 Registers 21
5.1.1 Overview 21
5.1.2 Common INC-DEC functionality problem 21
5.1.3 74LS193 Chaining 22
5.1.4 Circuit: Driver Module 22
5.1.5 Circuit: Data Register 23
5.1.6 Circuit: Data-Pointer Register 24
5.1.7 Circuit: Loop-Skip Register 25
5.1.8 Circuit: Instruction Register 25
5.1.9 Circuit: Instruction-Pointer Register 25
5.1.10 Circuit: Stack Register 25
5.1.11 Circuit: Flag Register 25
5.2 Memory 25
5.3 Control Unit 25
5.4 Input 25
5.5 Output 25

6 Algorithms **26**

1 Introduction

The Brainf*ck[1] (BF) programming language is an esoteric programming language that is basically impossible, or at least very unpractical, to actually write useful programs in. Even if you would become a very skilled programmer in this language, the resulting programs would be incredibly slow to execute. Despite this, many programmers have challenged themselves to write stunning pieces of code just for fun, or for the learning experience it offers. In doing so, it teaches us about computer architecture, compilers/interpreters, memory, pointers and much more. For more information on the language itself, see chapter 2.

The goal of this project is to build a computer that can actually run BF code natively. Normally, after having written some new piece of BF, the programmer must run this code in another program to either compile or interpret it to see whether it works. However, when looking more closely at the language, it seems a lot like an instruction set for a (not yet) existing processor. This is what we aim to do in this project: build that thing.

The aim is to build a Brainf*ck CPU without making use of any programmable chips. We will only use discrete logic and relatively simple integrated circuits like registers, buffers and (de)multiplexers. The computer will be built on breadboards, as it was inspired by Ben Eater's 8-bit breadboard computer [3]. This report will document the design and implementation of the BFCPU.

[1] The asterisk was inserted by the authors and is not part of the official name.

2 Brainf*ck

2.1 BF as a Language

Brainf*ck is often introduced as a programming language, which it is. However as we will soon see, it can also be viewed as an instruction set to a yet non-existent CPU. Just like any other programming language, it allows the programmer to write programs consisting of commands that are executed in order. There is a total of 8 commands available to the programmer, written as single characters: "`+-<>[].,`". Each of these commands corresponds to an operation on an array of memory or a pointer, pointing to some location within this memory. At the start of the program, every cell in memory is initialized to 0 and the pointer is pointing to the very first element (index 0, see Figure 1).

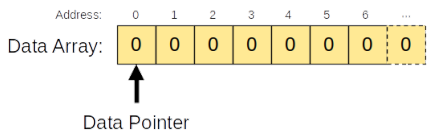

Figure 1: Initial state of the BF machine.

The commands then modify the contents of memory and the pointer as follows:

- `+` : add 1 to the current cell;
- `-` : subtract 1 from the current cell;
- `<` : move the pointer 1 cell to its left;
- `>` : move the pointer 1 cell to its right;
- `[` : if the current cell is nonzero, continue. Otherwise, skip to the matching closing `]`;
- `]` : if the current cell is zero, continue. Otherwise, loop back to its matching opening `[`;
- `.` : send the value in the current cell to the output;
- `,` : read a value from the input and store it into the current cell.

Although this might seem like a very limited set of instructions, it has been proven to be sufficient for performing any possible computation or program, also known as Turing-completeness [1]. The catch is that this requires an unbounded (or infinite) amount of memory, which is obviously impossible. However, the same caveat holds for traditional (von Neumann architecture) systems, so we should be safe to assume that BF is Turing complete for all practical purposes.

2.2 Architectures

2.2.1 Von Neumann

Modern computers are built according to the von Neumann architecture [2], which specifies a CPU (containing registers and an ALU), a single unit of memory and input/output devices (Figure 2). The registers of the CPU can be loaded with data from the memory unit and operated on by the ALU (Arithmetic and Logic Unit). Typical about this kind of architecture is the fact that not only data, but also the instructions (the program) are stored in memory. The program is therefore just as much part of the data as the data itself and can even be modified by itself.

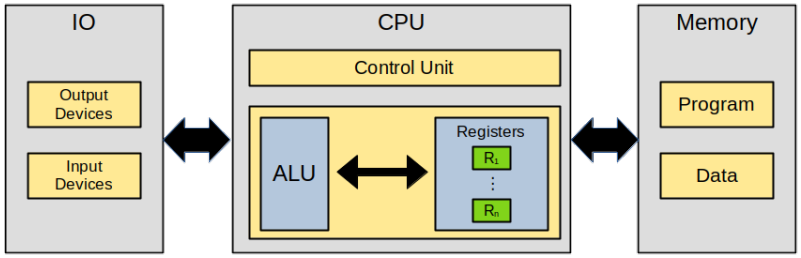

Figure 2: Schematic overview of the Von Neumann architecture.

2.2.2 Harvard

Unlike within the Von Neumann architecture, the Harvard architecture specifies two kinds of memory: program memory and data memory (Figure 3). The program memory contains only the instructions to be carried out and cannot be modified at runtime. Other than that, the architecture is similar to Von Neumann, in that it consists of a CPU (again containing registers and an ALU), memory (program and data) and input/output devices.

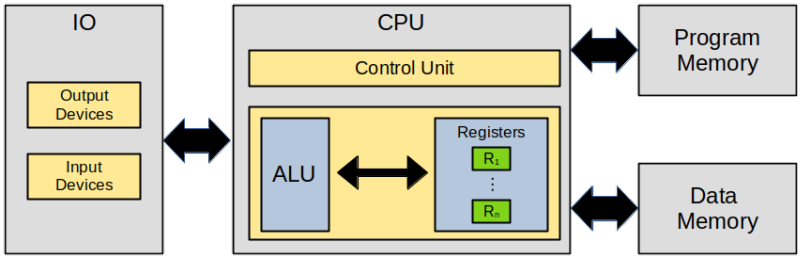

Figure 3: Schematic overview of the Harvard architecture.

2.2.3 BF Architecture

The architecture assumed by the BF language is similar to the Harvard architecture, in that the memory does not contain the program itself. This implies that the program is stored somewhere else and cannot be addressed by the pointer. The ALU is very limited and can only perform increment, decrement and comparison to zero.

2.3 BF as an Instruction Set

Instead of viewing BF as a language that needs to be compiled or interpreted on a traditional machine, it can also be seen as an instruction set to a processor, built according to the BF architecture described above. An instruction set of size 8 is truly tiny compared to more traditional instruction sets such as those implemented by modern processors or even microcontrollers and older 8-bit systems. Broadly speaking, Complex Instruction Set Computers (CISC) are designed to do as much work as possible in the least number of clock cycles, whereas Reduced Instruction Set Computers (RISC) focus on having a small instruction set with basic operations. For comparison, the x86 instruction set is massive with over 1500 instructions implemented in hardware, whereas RISC processors only need to implement 50 to 100 instuctions. Even compared to RISC, the BF instruction set (BFISC from hereon) is tiny even compared to the smallest instruction sets in use today. This isn't necessarily a good thing; a smaller number of instructions simply means you need more of them to perform meaningful computations, which is reflected by the fact that complex BF programs are typically very large in size.

2.4 Brainfix

In 2013, one of the authors wrote a compiler, Brainfix, that takes a human-readable programming language (also called Brainfix) and compiles this into BF. This project was then abandoned and rebooted in late 2021 with a complete rewrite of the compiler. While working on this project, the idea to build an actual computer to go along with the compiler arose. The Github project page holds an extensive manual to the Brainfix language and compiler; see [4]. Chapter 6 goes into detail about some of the BF algorithms implemented by the compiler.

3 Architecture

In simple terms, a BF machine consists of an array of memory-cells, together with a pointer pointing to one of these cells. The pointer can move along the array while modifying its contents one step at a time. An example of this representation in some intermediate state is shown in Figure 4. Consider the BF program "`>>>>>+.`", applied to the initial conditions shown in the example. The pointer would take 5 steps to the right, landing on cell 9 which contains the number 41. It will then increment and output this value, displaying 42 on the screen (assuming a screen of some sort is used as the output device and it is displaying numbers directly rather than interpreting them as ASCII).

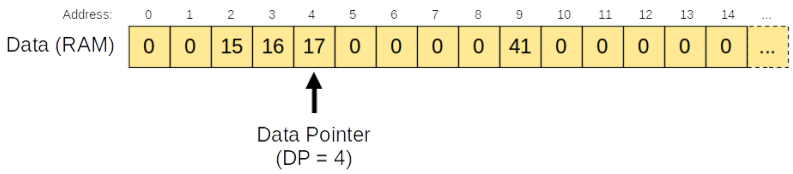

Figure 4: Example state of a BF machine.

In practice, a physical implementation of a machine like this needs many more components in addition to the RAM and Data Pointer in order to perform all the necessary operations. This chapter will list the modules that comprise the BF processor and give an overview of the way these modules function and communicate. The actual implementation on the logic/hardware level is described in section 5.

3.1 Overview

The processor consists of three basic building blocks: registers, memory and a control unit. The ALU is missing from this list because the only operations that it needs to perform are addition and subtraction of the value 1, which can be done directly at the register-level when using up/down binary counters like the 74LS193 integrated circuit. The program (a sequence of BF instructions) is stored into Read Only Memory (ROM), whereas the data is stored in Random Access Memory (RAM). Instructions (4-bits) are loaded from ROM into the instruction register (I), together with some flags that encode the state of the machine. Depending on the state and current instruction, the Control Unit sets the appropriate control signals for each of the modules in order for the system to perform the next computation. Figure 5 shows how each of the modules is communicating with other modules. In the sections below, each of these connections will be clarified further.

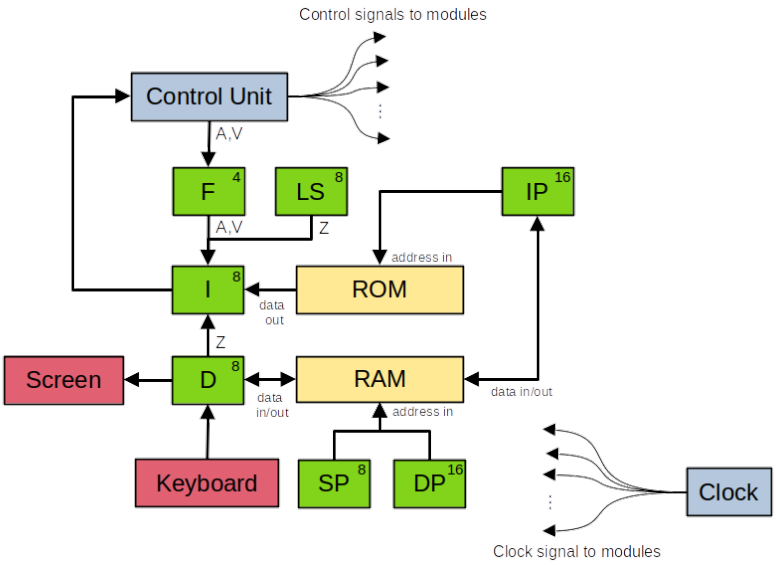

Figure 5: Connections between modules in the BF processor.

3.2 Instruction Pointer Register (IP)

The instruction pointer is a 16-bit value, kept in the IP register, which keeps track of the current instruction that is being executed. It points to a certain address in ROM (which stores the program) and is usually incremented after each instruction has finished executing, in order to move to the next instruction. However, when the processor encounters the `[`-instruction (and the loop is entered), it needs to store the current value of the IP somewhere in order to be able to jump back to this location. The value is therefore stored on the stack (the first part of RAM). When the matching `]`-instruction is encountered, this value is loaded back into the IP instead of simply incrementing the previous value. This has the effect of jumping back in the program, which is how loops are implemented in BF.

3.3 Stack Pointer (SP)

The stack is the first part of RAM (addresses 0x0000 - 0x00ff) which is reserved to keep track of addresses that might need to be jumped to. The stack-pointer (SP) is incremented whenever a new value is stored on the stack and decremented whenever a value is popped off the stack. In this implementation, the SP is an 8-bit value, which means that at most 256 different values can be stored onto the stack before it the SP overflows (wraps around back to 0) and starts overwriting previous values. This would happen if a BF program was loaded that has more than 256 nested `[]`-pairs. Although possible, it is very unlikely to happen for the simple programs we intend to run.

3.4 Instruction Register (I)

The instruction register is an 8-bit register that stores the current instruction, which was loaded from ROM according to the IP value. The BF-instruction itself is only 4 bits wide, which leaves another 4 bits for encoding the state of the machine, using flags. There are 4 flags that determine the state of the machine:

- Z(D): the zero-flag set by the D-register, indicating that there is currently a 0 stored in this register;
- Z(LS): the zero-flag set by the LS-register (see 3.7);
- A: the address-changed-flag, set by the control-unit, indicating that the previous instruction has changed the value of the data-pointer. When this flag is set, the value in the D-register does no longer correspond to the cell pointed to by the data-pointer;
- V: the value-changed-flag, set by the control-unit, indicating that the previous instruction has altered the value in the D-register. When this flag is set, the value in RAM is outdated and needs to be updated before moving the pointer to a different cell.

3.5 Data Register (D) and Data Pointer Register (DP)

The data-pointer corresponds to the pointer as specified in the BF-language. It points to some value in memory beyond the stack ($\geq$ 0x0100) and can be either incremented (moved right) or decremented (moved left) using the `>` and `<` instructions. Whenever the value pointed to by DP is modified by `+` or `-`, it is loaded into the D-register, where it can be modified before being stored back into RAM. This happens in conjunction with the flag-register, which dictates whether or not synchronization has to take place between RAM and D (see also section 3.6).

3.6 Flags and the Flag Register (F)

3.6.1 A and V

A naive and fail-safe way of synchronizing the contents of RAM with the contents of the D-register, is to perform the following sequence on each `+` or `-` instruction:

1. Load the current value (pointed to by DP) into D;
2. Modify the value;
3. Write the value back into RAM.

However, it is very common to have sequences of many repeating `+`'s or `-`'s and it would be a waste to keep writing the contents of D back to RAM, only to read them back into D during the very next instruction. This is why the

Control Unit sets a flag whenever either the value of the DP has changed (A, the address-changed-flag) or the value in the D-register has changed (V, the value-changed-flag). Depending on the state of these flags, the Control Unit can determine whether it has to synchronize the RAM with the D-register, or not (yet). The Control Unit buffers these flags in the flag-register (F), which is loaded into the instruction register (I) together with the next instruction from ROM.

3.6.2 Z(D) and Z(LS)

In addition to the A and V flags, there are two flags which are not buffered in F, but can be directly loaded into I: the zero-flags (Z) from the data-register and loop-skip-register (see section 3.7). These flags are both used in the context of conditional jumps:

- Z(D) : When encountering either an opening `[`, the flag indicates whether or not to enter or skip the loop. On a closing `]`, the flag indicates whether to loop back or exit from the loop.
- Z(LS) : The zero-flag from the loop-skip register indicates whether or not we are currently in the process of skipping an entire piece of code. This happens when the current value is 0 when encountering an opening `[`. In this case, execution must resume after the matching closing `]`. Only when the LS has its zero-flag set, should the current instruction actually be executed.

3.7 Loop Skip Register (LS)

The Loop Skip (LS) register is a counter that indicates whether or not we're in the process of sckipping a loop. In BF, a loop (`[`) is only entered when the value currently pointed to is nonzero. In the case that it is zero, execution resumes beyond its matching `]`. When it is determined (by the Control Unit) that a loop must be skipped (based on the zero-flag set by the D-register), the LS register is incremented from 0 to 1. Subsequent instructions are then skipped until either another (nested) loop opening `[` or a closing `]` is encountered. On the former, the LS is incremented again while on the latter the LS is decremented. This has the effect that the LS becomes 0 again after the `]` that matches the original `[` which led to the skip. Normal execution occurs as soon as LS has become 0 again.

3.7.1 Preprocessing

The LS-register is solving a problem that can be avoided by preprocessing the BF code. We could have chosen to somehow include jump-locations in the program and render scanning during runtime (keeping track of the counter to determine when to resume execution) unnecessary. This would however introduce its own set of issues but would also require a fundamental modification to

the BF instruction set. We therefore chose not to preprocess the program and require the computer to handle 'vanilla' BF.

3.8 Control Unit and Control Signals

Each of the aforementioned components/modules has one or more control inputs that determine what happens on the next clock cycle. For example, some register-modules can be told to either load a value from their input, increment or decrement the currently stored value, or do nothing at all. It is the Control Unit (CU) that supplies the appropriate control signals to each of the modules before the next clock pulse occurs. The implementation details of how this is done in hardware are discussed in section 5. The following sections will provide an overview of each of the control signals for each module. Section 4 will discuss the control sequences necessary to perform each BF instruction.

3.8.1 Control Unit

Even though the control unit is setting the control signals for all other modules, it has some control signals that it should be able to set for itself. Depending on the instruction that is being executed, it should write values to the flag register. The control signals to indicate that this is being done are called AE and VE respectively. Moreover, its internal instruction counter should be reset in the final cycle of each instruction. This is indicated with the CR (cycle reset) signal.

- AE: A-flag enable - outputs a high value into the A-bit of the flag register;
- VE: V-flag enable - outputs a high value into the V-bit of the flag register.
- CR: Cycle reset - reset the internal cycle counter to 0 to prepare for the next instruction.

3.8.2 Register Control Signals

Not all registers (green modules in Figure 5) have the exact same functionality. All of them store a value, but not all are required to count, let alone count in both directions. Table 1 shows which of the registers are able to perform certain actions. The control signals have been abbrivated as follows:

- EN: output enable - if this signal is unavailable, its output is either always available or not used at all;
- LD: load a value from the input;
- CE: count enable - increment the current value;
- DEC: if the CE signal is supplied, decrement instead of increment;

Register	#Bits	EN	LD	CE	DEC
D	8	x	x	x	x
I	8		x		
DP	16	x		x	x
SP	8	x		x	x
IP	16	x	x	x	
F	4		x		
LS	8			x	x

Table 1: Control signals available to each of the registers.

3.8.3 Memory (RAM)

The memory module has an address input and a set of I/O lines which can be used for either storing a new value into memory (input) or outputting the value stored at the current address. It is therefore sufficient to have only two control signals that go into the RAM module:

- OE: output enable - the value at the current address (supplied on the address lines) will be available at the output;
- WE: write enable - writes the value at the input to the current address.

3.8.4 Screen (Output)

The output module (which is assumed to be a screen) will be directly attached to the data-register and will display whatever is in there (on every clock pulse), when the PRE signal (print enable) is active.

- PRE: Print Enable - Display the contents of D on the next clock pulse.

3.8.5 Keyboard (Input)

The input device to the computer is assumed to be a keyboard of some sort, which contains a buffer from which some 8-bit value can be read. Immediately, a design choice has to be made: what happens if the buffer is empty? Either the program continues to run (immediate mode) or the program waits for something to appear in the buffer (buffered mode). The former is convenient for instance when playing games, while the latter is convenient for programs that require user input in order to continue meaningfully. Rather than deciding on either of these modes, we can just implement both. In the microcode table (Table 2, Section **??**) there are two variants of the `,` command listed (the other one denoted by `'`), each of which has a different binary representation. When assembling a BF program, we simply pick one or the other and the program should run in the appropriate mode: `,` for buffered and `'` for immediate mode.

The buffer of the module contains all zero's when nothing is present. When running in immediate mode, the programmer should programmatically determine if a value was read and how to act accordingly. In buffered-mode however,

after reading from the buffer, the control unit will use the Z-flag from the D-register to determine whether or not to continue. As long as the value in D is zero, control flow will loop around, trying to get a value from the input buffer.

The only control signal going in to the input-module should therefore be the enable-signal, which makes the contents of the buffer available to the input of the D-register.

- EN: Output Enable - enables the contents of the input buffer to be loaded into D.

4 Control Sequences

By setting the control signals as described in Section 3.8 appropriately, the modules can work together to perform each of the BF instructions. Table 2 shows the control sequences that are executed per BF instruction (also known as microcode instructions). The Control Unit implements this as a lookup table in ROM, where the instruction, flags and cycle counter act as an address into this table and the control signal configuration is stored at this address in ROM (Figure 6).

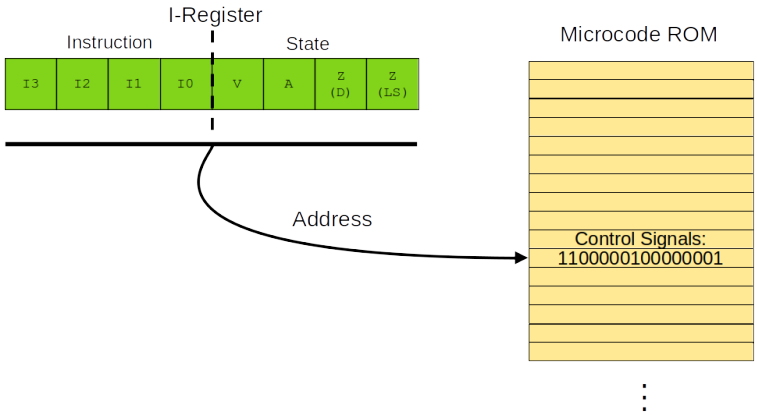

Figure 6: Decoding an instruction: the current instruction and state index the ROM, which returns the control signal configuration.

It is important to note that, because of the choice of driving all of the (counting) registers with a common interface (Section 5.1), only one register can be driven each clock cycle. That is, the CE and DEC signals can be applied to only one register in a clock cycle. Below we will go through each of the instructions in order to annotate the contents of Table 2.

4.1 Cylce 0

The first cycle of each instruction is identical. In fact, it's not really part of any instruction yet, because its purpose is to fetch the next instruction from ROM. In order to do so, the IP is enabled (address-in to the ROM module) and the resulting instruction is loaded into the I-register. Note that this also means that the A and V flags from the F register are loaded in (F is always enabled), together with the Z flags supplied by both the D and LS registers.

4.2 Modifying Data: + and -

The operations performed by the + command depend on the state of the system. If the A flag is not set (the address has not changed since the last instruction), this means that the value in D already corresponds to the current cell in memory. In that case, the CE signal for the D register is set for it to increment on the next clock pulse. In addition to this, the V-flag is set to indicate that the value in D has been changed, which is then loaded into the F register. In the next cycle, the CE is set for the IP register to load the next instruction and the cycle count of the CU is reset to 0.

On the other hand, when the A flag *was* set, we first need to fetch the new value from RAM by enabeling the DP register and loading the resulting value into D. From hereon, the next set of control signals is identical to that described above in the case where A was not set.

The control signals necessary to perform the - command are similar to those of the + command, the only difference being the DEC(D) signal to perform a subtraction rather than addition.

None of the actions above need to be performed when the Z(LS) flag is zero, which means that we're in the process of skipping a loop-block. In this case, we ignore the command and increment the IP immediately.

4.3 Moving the Pointer: < and >

Moving the datapointer around requires similar instructions compared to modifying the dataregister, the difference being that we increment or decrement the DP-register instead of the D-register. If the V-flag was set in the previous instruction, we need to write the updated value in the D-register back to RAM before moving the pointer. If not, we can immediately move the pointer. In either case do we need to set the A-flag in the F-register in order for the next instruction to take into account that the address has changed and the contents in the D-register are therefore not synchronized with the RAM. Like before, this instruction is ignored when the Z(LS) flag is zero.

4.4 Conditional Jumping: [and]

These are by far the most complicated instructions that require a lot of additional logic. Because the BFISC lacks a JMP-instruction where the argument holds the destination address, the computer has to store the address of the opening [-command in case it needs to loop back when the time comes. Also, if the loop is not entered, the LS-register is used to determine when execution resumes.

Loop Start: In the first scenario, where the D-register is up-to-date (A-flag not set) and its Z-flag is high, we can immediately conclude that this loop should be skipped over. Hence, the LS-register is incremented and the next instruction is loaded (to be ignored until the LS-register becomes 0 again).

In the second scenario, the A-flag is still not set but the Z-flag for the D-register is not set either, meaning that control should enter the loop. It takes 3 cycles to do so: increment the stack-pointer (cycle 1), write the current IP to this address on the stack (cycle 2) and move to the next instruction (cycle 3).

In scenario 3, the A-flag is set, which means that we should first load the new data from RAM into the D-register (cycle 1). After loading a new value into D, flags and cycle count are reset to 0 (without incrementing the instruction pointer). This means the same instruction is reloaded with updated flags on the next iteration, putting the system into either one of the states above.

In the case that we were already skipping code (Z(LS) low), we need to increment the LS-register once more to account for another pair of nested `[]`'s (cycle 1) and then continue to the next instruction (cycle 2).

Loop End: In the first scenario, which takes 2 cycles to execute, there is a known (synchronized) zero in the D-register. This means we can immediately choose to exit the loop. To do so, the stack-pointer is decremented (cycle 1) to point at the previous value on the stack. In cycle 2, the IP is incremented as usual.

If there is a known nonzero value in D (scenario 2: Z(D) is low), this means we must loop back to the IP stored on the top of the stack. This value is loaded into the IP-register by enabling the SP and RAM and setting the LD signal for the IP-register (cycle 1). On the second cycle, this new IP (pointing to a `[`) is incremented to re-enter the loop.

In the thrid scenario, the contents of D are not yet synchronized with the RAM (A-flag is set), so we first need to load it in. After loading the value into D, the flags and cycle counter are reset to put the system back into one of the previously defined states.

Finally, when already in the process of skipping a loop, the LS-register is decremented before moving to the next instruction.

4.5 Output: `.`

There are three states that need to be taking into account when implementing the output-command. In the first state, the value in D is already in sync with the RAM and can be sent to the output device immediately. The output of D is enabled and PRE signal for the screen is set, in addition to incrementing the IP to move to the next instruction. When A was set, we first load the value from RAM into D before sending its contents to RAM. Lastly, when Z(LS) is low, this instruction can simply be skipped.

4.6 Input: `,` and `'`

As mentioned before in Section 3.8.5, the architecture implements two versions of the input command, buffered (`,`) and immediate (`'`). Because buffered inputs are more common in most (BF) programs other than game-like applications, this is the default mode (even though it is more complicated).

Buffered Mode: In this mode, control flow is stuck in a loop, waiting for something nonzero to appear in its D-register. A nonzero value means something was read from the input module, whereas a zero value indicates that the input buffer was empty. When the V-flag is not set, the contents of D can be overwritten immediately by loading the keyboard-buffer into D. After doing so, the instruction is reloaded to update the Z(D) flag, which is then used to decide between either moving to the next instruction (when Z(D) is 0) or looping back to cycle 0 (without incrementing the IP). When the V-flag *is* set however, the current value stored in D must first be written to RAM in order to synchronize. After having done so, the cycle counter is reset to 0 to put the system back into the state above where the keyboard-buffer can be read into the D-register. Of course, when Z(LS) is zero, we can simply skip all of this.

Immediate Mode: In this mode, we don't care what was loaded into D, even if there was nothing there in the keyboard buffer; it is up to the programmer to handle the case where no keys were pressed. This makes the implementation a lot easier, taking at most 2 cycles to complete. In the first case, the V-flag is not set, so we can simply overwrite the contents of D by loading in whatever is in the keyboard-buffer and move to the next instruction. In the second case, where V *is* set, we write the contents of D to RAM before copying the keyboard buffer into D. Again, when Z(LS) is zero, this instruction is skipped.

4.7 Non-BF instructions: `NOP` and `ERR`

For debugging purposes, two non-BF instructions have been included. The `NOP` instruction does nothing. It simply increments the IP and resets the cycle count to move to the next instruction. The `ERR` instruction is inserted in all non-reachable states. If for some reason a state occurs that maps to the `ERR` command, the clock will be halted and some indicator on the Control Unit should light up to let its users know that something has gone wrong.

Instr	V	A	Z(LS)	Z(D)	Cycle	Control Signals			
Any					0	EN(IP)	LD(I)		
+		0	1		1	CE(D)	VE(CU)	LD(F)	
		0	1		2	CE(IP)	CR(CU)		
		1	1		1	EN(DP)	LD(D)	OE(RAM)	
		1	1		2	CE(D)	VE(CU)	LD(F)	
		1	1		3	CE(IP)	CR(CU)		
			0		1	CE(IP)	CR(CU)		
-		0	1		1	CE(D)	DEC(D)	VE(CU)	LD(F)
		0	1		2	CE(IP)	CR(CU)		
		1	1		1	EN(DP)	LD(D)	OE(RAM)	
		1	1		2	CE(D)	DEC(D)	VE(CU)	LD(F)
		1	1		3	CE(IP)	CR(CU)		
			0		1	CE(IP)	CR(CU)		
>	0		1		1	CE(DP)	AE(CU)	LD(F)	
	0		1		1	CE(IP)	CR(CU)		
	1		1		1	EN(D)	EN(DP)	WE(RAM)	
	1		1		2	CE(DP)	AE(CU)	LD(F)	
	1		1		3	CE(IP)	CR(CU)		
			0		1	CE(IP)	CR(CU)		
<	0		1		1	CE(DP)	DEC(DP)	AE(CU)	LD(F)
	0		1		2	CE(IP)	CR(CU)		
	1		1		1	EN(D)	EN(DP)	WE(RAM)	
	1		1		1	CE(DP)	DEC(DP)	AE(CU)	LD(F)
	1		1		3	CE(IP)	CR(CU)		
			0		1	CE(IP)	CR(CU)		
[		0	1	1	1	CE(LS)			
		0	1	1	2	CE(IP)	CR(CU)		

		0	1	0	1	CE(SP)					
		0	1	0	2	WE(RAM)	EN(SP)	EN(IP)			
		0	1	0	3	CE(IP)	CR(CU)				
		1	1		1	EN(DP)	LD(D)	OE(RAM)			
		1	1		2	LD(F)	CR(CU)				
			0		1	CE(LS)					
			0		2	CE(IP)	CR(CU)				
]		0	1	1	1	CE(SP)	DEC(SP)				
		0	1	1	2	CE(IP)	CR(CU)				
		0	1	0	1	EN(SP)	EN(RAM)	LD(IP)			
		0	1	0	2	CE(IP)	CR(CU)				
		1	1		1	EN(DP)	EN(RAM)	LD(D)			
		1	1		2	LD(F)	CR(CU)				
			0		1	CE(LS)	DEC(LS)				
			0		2	CE(IP)	CR(CU)				
.		0	1		1	PRE(SCR)	EN(D)	CE(IP)	CR(CU)		
		1	1		1	EN(DP)	EN(RAM)	LD(D)			
		1	1		2	PRE(SCR)	EN(D)	CE(IP)	CR(CU)		
			0		1	CE(IP)	CR(CU)				
,	0		1		1	EN(KB)	LD(D)				
	0		1		2	EN(IP)	LD(I)				
	0		1	0	3	VE(CU)	LD(F)	CE(IP)	CR(CU)		
	0		1	1	3	CR(CU)					
	1		1		1	EN(D)	EN(DP)	WE(RAM)			
	1		1		2	LD(F)	CR(CU)				
			0		1	CE(IP)	CR(CU)				
’	0		1		1	EN(KB)	LD(D)	VE(CU)	LD(F)	CE(IP)	CR(CU)
	1		1		1	EN(D)	EN(DP)	WE(RAM)			

	1	1	2	EN(KB)	LD(D)	VE(CU)	LD(F)	CE(IP)	CR(CU)
		0	1	CE(IP)	CR(CU)				
`NOP`			1	CE(IP)	CR(CU)				
`ERR`				ERR(CU)	HLT(CLC)				

Table 2: Control signals for each of the BF instructions in different scenario's, depending on the state flags. Note that in order to distinguish between the two input modes, the regular comma (`,`) and the apostrophe (`'`) are used for buffered and immediate inputs respectively. See also Section 3.8.5.

5 Implementation

5.1 Registers

This chapter will explain the implementation details of the register-module: the architecture of the driver module, data-pointer register, data, stack, loop-skip, instruction, instruction-pointer, and flag register. At the end of this chapter, we will provide the entire picture of the register-module.

5.1.1 Overview

In BFCPU, we omitted ALU because the only mathematical operation that brainf*ck can perform is adding and subtracting one. For the sake of simplicity, we decided to utilize 74LS193 and 74LS161 integrated chips to make individual registers. Both chips provide memory functionality along with incrementing of the value. Additionally, 74LS193 provides decrementing of the current value stored in the chip. It is important to note that F-R does not need any increment-decrement (INC-DEC) functionality. Thus, we used 74LS*** since it provides only the storage of a 4-bit value.

5.1.2 Common INC-DEC functionality problem

Register module possesses a lot of INC and DEC pins that the control unit needs to manipulate in order to alter the value of registers. Driver module removes this inconvenience and serves as the common interface for all registers, and thus the entire register-module. The idea behind driver module is limiting all INC and DEC pins to one DEC pin and one CE (count enable) pin, along with CLK (clock). If CE is HIGH, then based on the value of DEC pin, we can either decrement or increment on the rising edge of the clock pulse (CLK). The output of the driver module is the pair UP-DOWN that must be transmitted to an appropriate register[2]. Hence, we need a selecting interface to choose what register the computer is dealing with. The inner circuitry (see figure **??**) accomplished this goal. A very general overview of the register-module is shown in the figure below. There are three select pins that provide us with 8 different possible paths, which is sufficient because we have less than 8 different registers.

[2]Or just UP signal for registers with 74LS161, as this chip can only increment

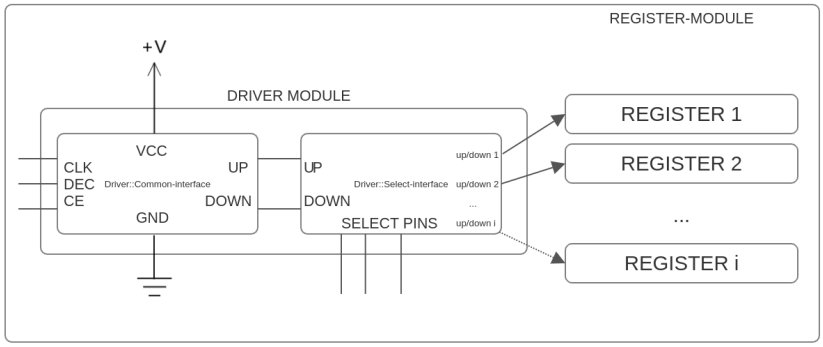

Figure 7: Register-module Overview

5.1.3 74LS193 Chaining

Registers use 74LS193 (or 74LS161) to obtain storage and INC-DEC functionality. However, one 74LS193 chip has 4 bits whereas registers require 8 and 16 bits. Fortunately, 74LS193[3] has a chaining ability: if CO (carry out) is connected to the UP pin of the other chip, and BO (borrow out) is connected to the DOWN pin, then we will obtain more bits available (see figure 8). This technique is used in every register that requires more than 4 bits.

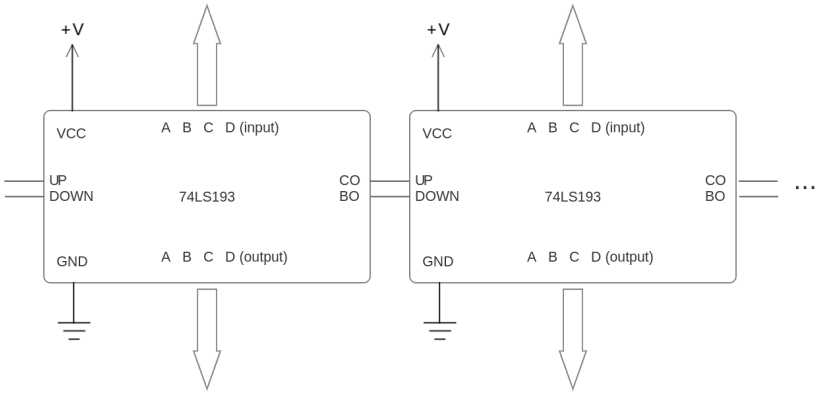

Figure 8: Register-module 74LS193 chaining

5.1.4 Circuit: Driver Module

As visible from figure 7, the driver module has two parts: common interface, select interface. Connection to registers is the bridge between the driver module and registers in the register module.

We used two demuxers to create the common interface. The basic idea behind this implementation is the following: DEC pin defines the way where

[3]Similar configuration works out for 74LS161, so we will not discuss it here

CE and CLK signals are propagated, so DEC is connected to the select pin of both demuxers. CE switches the signals off or on, while CLK stimulates the alterations in UP/DOWN pairs, contributing to the change in the value of a register.

Because of the inner circuit of 74LS193, we need to avoid simultaneous pulsing and UP/DOWN pair grounded since it unfavorably alters the value of a register. The solution was quite straightforward: CLK pin of the driver module is connected to the output of anded clock signal and CE pin.

The resulting UP/DOWN pair is directed to the select interface, that redirects the pair to a specific, selected, register. This behavior is accomplished by utilizing 3-to-8 demuxers, because we have less than 8 registers but more than 4.

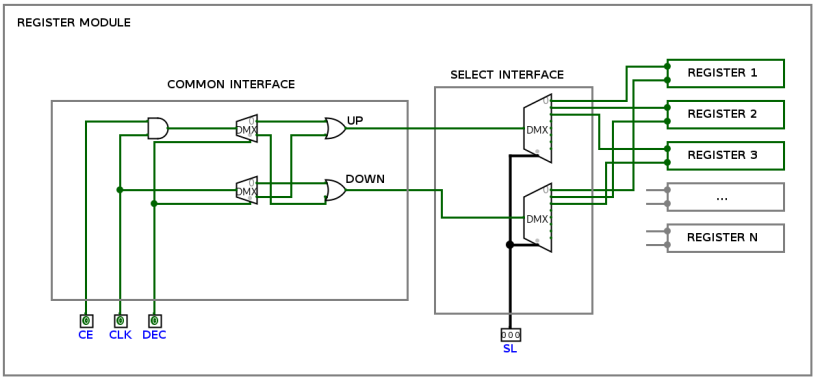

Figure 9: Driver Module Implementation

5.1.5 Circuit: Data Register

Data register is a 8-bit valued flip-flop with both increment and decrement functionality. Therefore, we used 74LS193 chip. One essential property of the data register is that it also outputs a so-called zero flag (Z), which is 1 only if stored value is 0. The latter ability is implemented using OR gates and direct connection to stored bits. (see figure 10)

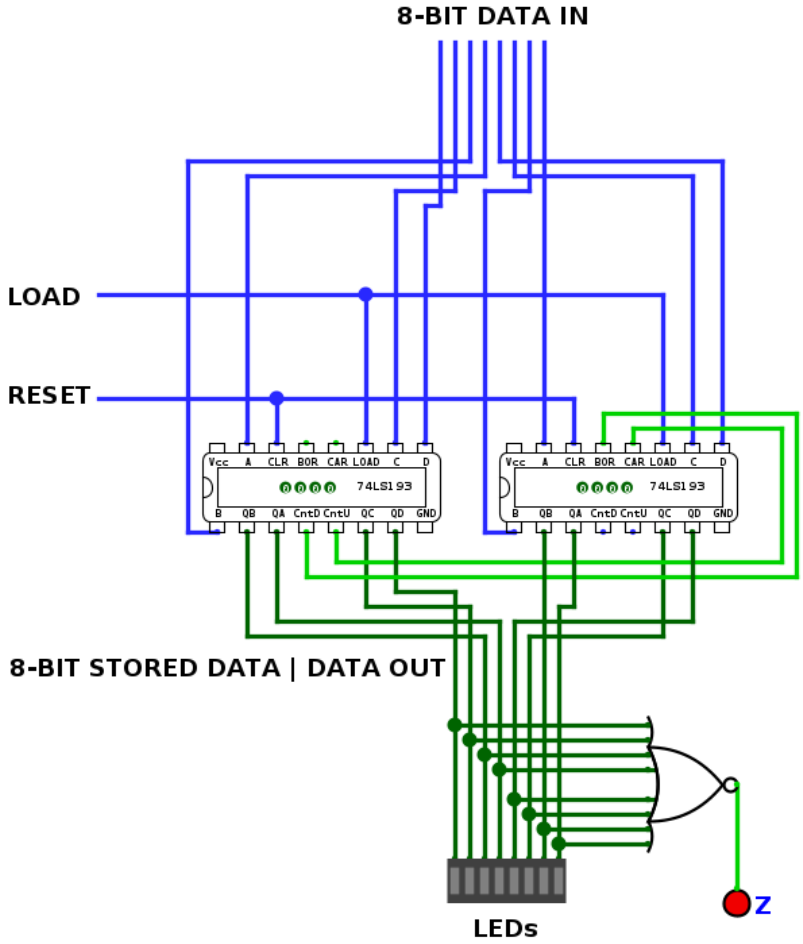

Figure 10: Data, and Loop-Skip Register Implementation (5.1.7)

5.1.6 Circuit: Data-Pointer Register

Data-Pointer register is a 8-bit valued flip-flop with both increment and decrement functionality. Therefore, we used 74LS193 chip. There is nothing very special about the data-pointer register, except that its output (value stored inside) is connected to the bus transceiver, but this applies for other registers as well[4]. (see figure 11)

[4]Implementation of the bus transceiver is fairly straightforward: as only outputs are connected along with DIR and CE (chip enable) pins as an input. Chips used is 74LS245

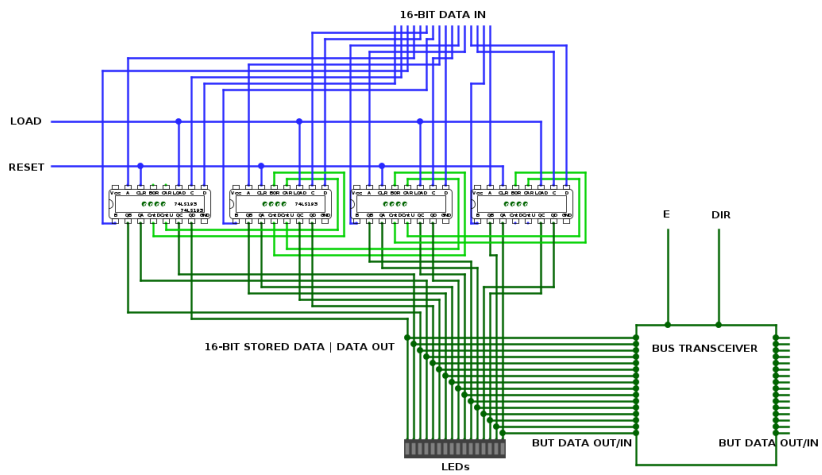

Figure 11: Data-Pointer Register Implementation

5.1.7 Circuit: Loop-Skip Register

Fundamentally, from the inner-circuit perspective loop-skip register (LS-R) has entirely the same implementation as the data-register (D-R): storage capacity of 8 bits, increment and decrement facilities, along with loop-flag[5]. (refer to figure 10)

5.1.8 Circuit: Instruction Register

5.1.9 Circuit: Instruction-Pointer Register

5.1.10 Circuit: Stack Register

5.1.11 Circuit: Flag Register

5.2 Memory

5.3 Control Unit

5.4 Input

5.5 Output

[5]Loop-flag is the same as zero-flag for data-register, thus is implemented the same way as in the data register

6 Algorithms

The BFX compiler implements a series of algorithms to perform basic operations on the data stored in the data-array, which map to common operators and statements in most programming languages. This section will document each of these algorithms, starting with the most fundamental ones, which we can then use to implement the more complex operations. In the process, we will develop a pseudo language that maps directly to BF-code. The statements of the pseudo language will always be written within a set of curly braces, in order to seperate them from the BF operators.

6.1 Moving the Pointer: {x}

Whenever we need to operate on a cell, we first need to move the datapointer to this cell. It is assumed that the compiler is aware of the current pointer position and will therefore be able to move the pointer to any other cell, as long as the address of this cell is also known at compile-time. From now on, the operation of moving to a cell designated by some variable `x` will be written in as {`x`}.

In future expressions enclosed by curly braces, the pointer is always left implicitly at the address of the left-hand-side operand.

6.2 Setting a Fixed Value: {x <- n}

The next elementary operation is setting some cell to a specific (known) value. We will not assume any prior knowledge of the current contents of this cell, which means the first step is to clear it completely, which we can do by iteratively decrementing the cell until it becomes zero: `[-]`. We then follow it up by the desired amount of increments. For example, setting cell `x` to `5` can be done with:

```
{x} [-] +++++
```

From now on, we will use the following shorthand for setting a cell (`x`) to a fixed value (n): {`x <- n`}.

6.2.1 Using + Instead: {x <- n+}

Instead of decrementing the cell until it becomes zero, we could also choose to increment it using `[+]`. The cell will eventually overflow and wrap back to zero, achieving the same result. This might seem unnecessary and dangerous, especially for 16-bit or larger values, because it can take a lot of time before the cell overflows. However, for some algorithms, where a cell is intentionally being *underflown*, this is a helpfull tool to have at hand. When we need it, we will use a slightly different shorthand: {`x <- n+`}.

6.3 Moving Data: {x <- y}

It turns out that moving data is a lot easier than copying data. When moving data from one cell to another, the source-cell will be left empty. Below

is a listing for the algorithm that moves the contents of cell x into cell y, leaving cell x empty after the move:

```
{y <- 0}
{x} [ {y} + {x} - ]
```

After clearing `y`, we move the pointer to `x` and decrement it until it becomes zero, while at the same time incrementing cell `y`. This algorithm can be extended to move the data into arbitrarily cells, simply by adding variables to the loop and incrementing those as well. Moving data from `x` to `y, z, ...` will from now on be written as {`(y, z, ...) <- x`}, analogous to moving a fixed value into a cell.

6.4 Assignment/Copy: {`x = y`}

An assignment of the form `x = y` will assign the contents of cell `y` to cell `x`, overwriting the previously present value of `x`. This can be achieved by moving the contents of `y` into both `x` and a temporary value `tmp`. This will leave `y` empty, but we can restore it by moving the contents of `tmp` back into `y`.

```
{(x, tmp) <- y}
{y <- tmp}
```

The shorthand for this operation will from now on be: {`x = y`}

6.5 Addition

6.5.1 In-Place Addition: {`x += y`}

It turns out that adding some value to a cell looks just like a copy, but without resetting the target cell first. Below is the pseudocode for adding the contents of `y` to `x`, leaving `y` intact.

```
{tmp <- 0}
{y} [ {x} + {tmp} + {y} - ]
{y <- tmp}
```

This operation will be denoted as {`x += y`}.

6.5.2 Return Variable: {`z <- x + y`}

Rather than adding the contents of a cell to another one and changing the value of the cell on the receiving end, we should also have an algorithm that stores the sum in a new cell, leaving both operands untouched (or at least unchanged). This can be achieved simply by copying the contents of one of the cells to a third cell and applying the addition algorithm:

```
{z = x}
{z += y}
```

6.5.3 Subtraction

Subtraction works exactly analogously to addition: we just replace the + with a -. This leads to the following shorthands:

- {`x -= y`} for subtracting a cell in-place and
- {`z = x - y`} for storing the difference in a return-variable.

6.6 Multiplication

Multiplication can be implemented as repeated addition, for which we already have the tools. The same strategy will be used as before, where we first develop an algorithm aimed at multiplying a cell in-place.

6.6.1 In-Place Multiplication: `x *= y`

To multiply `x` by the value stored in `y`, we need to add `x` to itself, `y` times. To achieve this, we copy the value of `y` to a temporary cell, which we can decrement while adding a copy of the original value of `x` to itself:

```
{xCopy = x}
{yCopy = y}
{yCopy}
[
  {x += xCopy}
  {yCopy} -
]
```

6.6.2 Return Variable: {`z <- x * y`}

Like before, we can copy the value of one of the operands into the return address and apply the in-place multiplication algorithm:

```
{z = x}
{z *= y}
```

6.7 Logical Operators

6.7.1 NOT: {`x <- NOT y`}

The NOT operator basically checks whether the argument is 0, in which case it returns 1. In all other cases, it should return a zero. This can be implemented using the loop-operators, which are the only conditional instructions available to us in BF.

```
{x <- 1}
{yCopy = y}
[
  {x      <- 0}
```

```
    {yCopy <- 0}
  ]
```

6.7.2 AND: {z <- x AND y}

The AND operator will only return 1 when both arguments (x and y) are nonzero:

```
  {z <- 0}
  {yCopy = y}
  {xCopy = x}
  [
    {yCopy}
    [
      {z     <- 1}
      {yCopy <- 0}
    ]
    {xCopy <- 0}
  ]
```

6.7.3 OR: {z <- x OR y}

The OR operator returns a 1 when either of its arguments are nonzero:

```
  {z <- 0}
  {xCopy = x}
  [
    {z     <- 1}
    {xCopy <- 0}
  ]
  {yCopy = y}
  [
    {z     <- 1}
    {yCopy <- 0}
  ]
```

6.7.4 Greater Than: {z <- x > y}

To check whether x has a higher value than y, we simply decrement x until it becomes 0. At each step, we also decrement y and check whether it is being decremented beyond zero (check for underflow). If that happens, this means that x is indeed the biggest.

```
  {z <- 0}
  {underflow <- 0}
  {yCopy = y}
```

```
{xCopy = x}
[
  {underflow <- NOT yCopy}
  {yCopy} -
  {z <- z OR underflow}
  {underflow}
  [
    {yCopy <- 0+}
    {underflow <- 0}
  ]
  {xCopy} -
]
```

6.7.5 Less Than: {`z <- x < y`}

The less than algorithm can just be expressed as a greater than algorithm, for which the arguments have been swapped around. It will therefore not be repeated below.

6.7.6 Equals: {`z <- x == y`}

Once the 'greater than' and 'less than' operators have been properly defined, the 'equals' operator can be defined in terms of these:

```
{z <- {NOT x < y} AND {NOT (x > y)}}
```

6.8 Division and Modulo: {`(div, mod) <- x / y`}

Division and modulo are done in a single algorithm. Division is implemented as a series of subtractions, counting how many times the numerator fits in the denominator. Two special cases are handled:

1. When the numerator is zero, the division algorithm is skipped and zero is returned.

2. When the denominator is zero, the algorithm is skipped and the maximum cell value is returned (which is as close to infinity as we can get).

To indicate whether either of these special cases was handled, we set a flag (the loop-flag) which is reset while handling these cases. Only when this flag is still set will the division algorithm be executed. Below is the pseudocode for the expression {`(div, mod) = x / y`}, where `div` is the result of the division and `mod` is the remainder after the division.

```
{div <- 0}
{mod <- 0}
{zeroFlag <- 1}
{loopFlag <- 1}
```

```
# Special case 1: denominator is 0
# ==> return max value (255 on 8-bit arch)
{zeroFlag <- NOT y}
[
  {div <- 255}
  {mod <- 255}
  {loopFlag <- 0}
  {zeroFlag <- 0}
]

# Special case 2: the numerator is 0
# ==> return 0 (even in 0/0 case)
{zeroFlag <- NOT x}
[
  {div <- 0}
  {mod <- 0}
  {loopFlag <- 0}
  {zeroFlag <- 0}
]

# Division algorithm: repeated subtraction
{xCopy = x}
{yCopy = y}

{loopFlag}
[
  {xCopy} -
  {yCopy} -
  {mod}   +

  # yCopy became 0?
  # ==> increase result by one and reset yCopy
  {zeroFlag <- NOT yCopy}
  [
    {div} +
    {mod <- 0}
    {yCopy = y}
    {zeroFlag <- 0}
  ]

  # xCopy became 0?
  # ==> done
  {zeroFlag <- NOT xCopy}
  [
    {loopFlag <- 0}
    {zeroFlag <- 0}
```

```
    ]
    {loopFlag}
  ]
```

References

[1] https://esolangs.org/wiki/Brainfuck

[2] https://en.wikipedia.org/wiki/von_neumann_architecture

[3] Ben Eater, *Build an 8-bit computer from scratch*, https://eater.net/8bit

[4] Joren Heit, *Brainfix*, https://github.com/jorenheit/brainfix

[5] Hitachi, *74LS193 Datasheet*, https://pdf1.alldatasheet.com/datasheet-pdf/view/64016/HITACHI/74LS193.html